WECAN ACADEMY • ARTIFICIAL INTELLIGENCE COURSE • SEASON 11

CBC Senior School Selection & Career Advisory Tool

A Two-Stage Generative AI CLI Decision Assistant for Grade 9 CBC Transitions

GROUP NAME	**THE LIONS**
GROUP MEMBERS	Ronny Ajanja • Kennedy Chomba Gitari • Angela Njeri Kiguru • Caroline Gathigia Mwangi
GITHUB REPOSITORY	`https://github.com/ronnyajanja-stack/wca-ai-tool-s11-The-Lions`
PROJECT TITLE	AI-Powered Decision Assistant • End of Module Group Project (60 Marks)
DATE OF SUBMISSION	August 2026

1. Problem Statement

Under Kenya's Competency-Based Curriculum (CBC), Grade 9 learners stand at a critical academic juncture as they prepare to transition into Senior School (Grades 10, 11, and 12). Senior School is differentiated into three primary pathways: *STEM (Science, Technology, Engineering, and Mathematics)*, *Social Sciences (Humanities and Business)*, and *Arts & Sports Science*. Each pathway encompasses specialized subject combinations that directly dictate post-secondary placement and career opportunities.

Currently, Grade 9 learners, parents, and school guidance personnel face significant decision friction:

- **Fragmented & Opaque Information:** Guidelines regarding which subject combinations support specific pathways and regional institutional capacities are scattered and hard to evaluate comparatively.
- **Disconnect Between Selection and Long-term Careers:** Families often select subjects without understanding their downstream alignment with Kenyan university degree clusters, TVET programs, or dynamic labor market demands.
- **Information Asymmetry:** Families outside urban centers often lack access to professional vocational counselors to interpret pathway implications.

Beneficiaries: The primary beneficiaries are Kenyan Grade 9 learners, parents/guardians, and junior school career guidance masters. The tool serves as an interactive advisory copilot that democratizes access to structured, contextualized guidance while strictly adhering to Kenyan CBC framework guidelines.

2. Tool Description

The **CBC Senior School Selection & Career Advisory Tool** is a command-line interface (CLI) application engineered in Python. It guides users through an interactive, two-stage AI advisory workflow designed to turn subjective interests into concrete academic and career decisions.

The application presents an intuitive main menu with three core options:

1. **Senior School Matching & Pathway Alignment (Stage One)**
2. **Career Opportunity & Industry Insights (Stage Two)**
3. **Export Guidance & Exit**

Stage One: Senior School Matching & Pathway Alignment

The user enters their intended CBC pathway (e.g., STEM - Pure Sciences), preferred county/region (e.g., Kiambu / Central), and subject combination interests (e.g., Physics, Chemistry, Biology, Advanced Mathematics). The application validates these inputs, sends a structured R-T-C-C-O prompt to the LLM, and enforces strict JSON output. The parsed response presents:

- **Pathway Compatibility Assessment:** How well the chosen subjects match CBC requirements.
- **Regional & Institutional Fit:** Criteria of schools equipped for the chosen pathway in that locality.
- **Advisory Next Steps & Verification Flags:** Clear highlights of facts requiring validation with official Ministry of Education / KNEC circulars.

Stage Two: Career Opportunity & Industry Insights

Stage Two leverages state management from Stage One. The selected pathway and learner profile are injected into the second AI prompt. The model delivers an in-depth Markdown roadmap detailing:

- **Target Career Clusters & Emerging Roles:** Professional disciplines aligned to the CBC pathway in Kenya and Eastern Africa.
- **Post-Secondary Progression:** Direct university degree routes, TVET diploma options, and professional certifications.
- **Core Competency Roadmap:** Key technical and soft skills to develop during Senior School.
- **Local Industry Demand Context:** Actionable next steps and extracurricular growth areas.

3. AI Instruction Design (R-T-C-C-O Framework)

Both API requests are engineered using the **R-T-C-C-O** (Role, Task, Context, Constraints, Output) prompt design standard to guarantee high precision, zero hallucinations regarding admissions data, and programmatic reliability.

3.1 API Call #1: School Selection & Compatibility Analysis (JSON Output)

```
ROLE: You are an expert Kenyan CBC educational advisor and senior school placement consultant.
TASK: Analyze the learner's preferred CBC pathway, location/county, and subject combinations
to evaluate institutional fit, subject synergy, and preparation requirements.
CONTEXT: A Kenyan Grade 9 learner is transitioning to Senior School and requires structured,
actionable guidance to make informed subject and pathway choices.
CONSTRAINTS:
- Strictly align recommendations with official Kenya CBC pathway guidelines (STEM, Social
Sciences, Arts & Sports).
- Use realistic Kenyan geographic regions and educational contexts.
- DO NOT hallucinate school rankings, exact cutoff marks, guaranteed vacancies, or specific
school fee structures.
- Flag any administrative facts that must be confirmed via official Ministry of Education/KNEC
channels.
OUTPUT: Return VALID JSON ONLY (no markdown fences, no conversational preamble) matching this
schema:
{
  "learner_profile": {"pathway": "...", "location": "...", "subjects": "..."},
  "pathway_alignment": {"status": "Strong/Moderate/Caution", "summary": "..."},
  "regional_considerations": "...",
  "recommended_school_characteristics": ["...", "..."],
  "subject_fit_analysis": "...",
  "actionable_next_steps": ["...", "..."],
  "verification_notes": ["...", "..."]
}
INPUTS:
- Preferred Pathway: {pathway}
- County/Location: {location}
- Selected Subjects: {subjects}
```

Design Justification: Enforcing pure JSON output allows the Python backend to validate structural integrity with schema enforcement before displaying the guidance. Explicit constraints prevent the model from fabricating real-time school fee amounts or official placement quotas.

3.2 API Call #2: Career Opportunity & Industry Advisory (Markdown Output)

```
ROLE: You are a senior vocational guidance counselor and East African labor market analyst.
TASK: Generate a comprehensive, forward-looking career roadmap and industry analysis based on
the learner's validated CBC pathway.
CONTEXT: A Grade 9 graduate and their guardians need long-term visibility into post-secondary
routes, labor demand, and vocational competencies following senior school.
CONSTRAINTS:
- Keep language age-appropriate, empowering, and realistic.
- Do not guarantee employment, salary figures, or university admissions.
- Distinguish clearly between academic university degrees and technical/TVET vocational
routes.
- Focus on practical Kenyan and regional market realities.
OUTPUT: Return clean, well-formatted Markdown with the following exact section headers:
# CBC Pathway Career Roadmap: {pathway}
## 1. Pathway Overview & Strategic Value
## 2. Key Career Clusters & Professional Roles
## 3. Post-Secondary Education & Training Routes (University & TVET)
## 4. High-Priority Competencies & Practical Skills
## 5. Kenyan Labor Market & Regional Industry Insights
## 6. Recommended Extracurricular & Actionable Next Steps
INPUT:
- Validated Pathway from Stage 1: {pathway}
```

> **Design Justification:** Markdown output is ideal for reading and direct export. Passing the confirmed pathway state from Stage 1 ensures contextual continuity without requiring the user to re-type their data.

4. Technical Overview

The tool is structured modularly following clean software engineering principles. The architecture decouples CLI flow control, API client management, prompt definitions, schema validation, and file exporting.

Module	Architectural Responsibility
`main.py`	Coordinates user input loops, manages session state, controls menu navigation, and handles safe shutdown.
`prompts.py`	Encapsulates the parameterized R-T-C-C-O prompt templates for Stage One (JSON) and Stage Two (Markdown).
`ai_client.py`	Manages OpenAI client instances, API keys via environment variables, model dispatch, and network exception trapping.
`parsers.py`	Parses and validates LLM JSON responses, ensuring required keys exist before passing data to the UI layer.
`exporter.py`	Aggregates session records and writes timestamped, clean Markdown advisory reports to an `exports/` directory.
`.env` / `.gitignore`	Secures sensitive API credentials and prevents accidental commits of secrets and local artifacts to GitHub.

Two-Stage Data Flow & State Integration

The core workflow maintains session state across API calls. The output of Stage One dynamically populates Stage Two:

```
# Stage 1: Execute School & Pathway Compatibility Analysis
raw_school_response = ask_ai(school_prompt(pathway, location, subjects))
parsed_school_data = parse_school_result(raw_school_response)
display_school_guidance(parsed_school_data)

# Extract validated pathway state (falls back to user input if omitted in payload)
selected_pathway = parsed_school_data.get("learner_profile", {}).get("pathway", pathway)

# Stage 2: Ingest Stage 1 Pathway State into Career Advisory Engine
career_prompt_text = career_prompt(selected_pathway)
career_report_markdown = ask_ai(career_prompt_text)
display_career_guidance(career_report_markdown)

# Session Preservation: Export structured results to local disk
export_path = export_session(parsed_school_data, career_report_markdown)
```

5. Testing & Error Handling

To guarantee resilience against real-world failures, comprehensive defensive programming was implemented across all interaction boundaries:

Failure Scenario Tested	What Broke During Development	Implemented Defensive Mechanism
Blank / Whitespace User Input		

Failure Scenario Tested	What Broke During Development	Implemented Defensive Mechanism
	Empty strings passed into prompt templates produced degraded, overly generic AI responses.	Implemented a `required(prompt_text)` input validation loop that refuses empty strings and re-prompts the user.
Invalid Menu Selection	Non-numeric inputs or integers out of range crashed or stalled the CLI loop.	Encapsulated menu dispatcher in a controlled `while` loop with polite error guidance.
Network Drop / API Timeout	Raw `APIConnectionError` / HTTP exceptions resulted in unhandled Python stack traces.	Wrapped API calls in robust `try...except Exception` blocks in `ai_client.py`, presenting clear retry messages.
Malformed JSON from AI	Model occasionally included Markdown backticks (```` ```json ````), breaking `json.loads()`.	Added automated markdown-fence stripping and schema validation verifying required keys in `parsers.py`.
Missing / Unset API Key	Application raised unhandled `AuthenticationError` on startup.	Pre-flight environment validation in `ai_client.py` checks `os.getenv("OPENAI_API_KEY")` and warns the user clearly.
Premature Stage 2 Access	User selecting Option 2 before Option 1 caused `NoneType` errors in prompt generation.	State gate checks whether `pathway` is populated; if not, instructs user to complete Stage 1 first.

6. Ethics, Privacy & Responsible AI Reflection

- **Mitigating Bias in Career Guidance:** Generative models may carry training biases favoring traditional careers (e.g., medicine, law) over vocational, technical (TVET), or creative arts pathways. Our R-T-C-C-O constraints mandate balanced parity across technical, creative, and academic trajectories to avoid pigeonholing learners.
- **Data Privacy & Minimal Collection:** In alignment with responsible AI principles, the application does not ask for or store personally identifiable information (PII) such as student full names, National ID/NEMIS numbers, or specific school names. It operates strictly on abstract preference parameters.
- **Safe & Transparent Disclaimers:** Generated recommendations are clearly presented as decision-support guidance rather than statutory admissions placements. The tool explicitly instructs parents and learners to verify fees, vacancies, and prerequisites through official Kenya Ministry of Education and KNEC announcements.
- **API Key & Codebase Security:** Strict `.gitignore` rules prevent credentials from leaking to public GitHub repositories.

7. Conclusion & Future Improvements

The **CBC Senior School Selection & Career Advisory Tool** successfully demonstrates how a structured, two-stage conversational AI system can bridge information gaps during crucial educational transitions in Kenya. By chaining structured JSON analytical outputs into contextualized career insights, the tool delivers immediate, practical value to Grade 9 learners and their guardians.

Roadmap for Future Enhancements:

- **Verified RAG Knowledge Base:** Integrate a vector database of official Ministry of Education Senior School catalogs, cluster guidelines, and KNEC regulations via Retrieval-Augmented Generation (RAG).
- **Multilingual Localization:** Expand system prompts to support *Kiswahili* and indigenous Kenyan languages for nationwide accessibility.

- **Graphical Web & Mobile Interface:** Transition from CLI to a responsive Streamlit or Flutter web app for streamlined mobile smartphone access.
- **Automated PDF Report Generator:** Replace basic Markdown exports with automated, branded PDF summary downloads.

8. Appendix: Full Project Source Code

Below is the complete, documented reference source code repository for the application.

8.1 .gitignore

```
# Environment configuration
.env
.env.local

# Python runtime artifacts
venv/
__pycache__/
*.pyc

# Generated session exports
exports/
*.json
*.md
*.txt
```

8.2 .env.example

```
# OpenAI API Configuration
OPENAI_API_KEY=your_openai_api_key_here
OPENAI_MODEL=gpt-4o-mini
```

8.3 prompts.py

```
"""
prompts.py - Prompt engineering module utilizing R-T-C-C-O framework.
"""

def school_prompt(pathway: str, location: str, subjects: str) -> str:
    """
    Builds the Stage 1 prompt for senior school matching.
    Enforces strict JSON output schema.
    """
    return f"""ROLE: You are an expert Kenyan CBC educational advisor and senior school placement
consultant.
TASK: Analyze the learner's preferred CBC pathway, location/county, and subject combinations to
evaluate institutional fit, subject synergy, and preparation requirements.
CONTEXT: A Kenyan Grade 9 learner is transitioning to Senior School and requires structured, actionable
guidance.
CONSTRAINTS:
- Strictly align recommendations with official Kenya CBC pathway guidelines (STEM, Social Sciences,
Arts & Sports).
- Use realistic Kenyan geographic regions and educational contexts.
- DO NOT hallucinate school rankings, exact cutoff marks, guaranteed vacancies, or specific school fee
structures.
- Flag any administrative facts that must be confirmed via official Ministry of Education/KNEC
channels.
OUTPUT: Return VALID JSON ONLY (no markdown formatting, no code fences) matching this schema:
{{
  "learner_profile": {{"pathway": "{pathway}", "location": "{location}", "subjects": "{subjects}"}},
  "pathway_alignment": {{"status": "Strong/Moderate/Caution", "summary": "Detailed alignment
summary"}},
  "regional_considerations": "Key institutional capacity notes for the region",
  "recommended_school_characteristics": ["Characteristic 1", "Characteristic 2"],
  "subject_fit_analysis": "Explanation of subject synergy",
```

```
  "actionable_next_steps": ["Step 1", "Step 2"],
  "verification_notes": ["Note 1", "Note 2"]
}}
INPUTS:
- Preferred Pathway: {pathway}
- County/Location: {location}
- Selected Subjects: {subjects}"""


def career_prompt(pathway: str) -> str:
    """
    Builds the Stage 2 prompt for long-term career roadmap generation.
    Outputs structured Markdown.
    """
    return f"""ROLE: You are a senior vocational guidance counselor and East African labor market
analyst.
TASK: Generate a comprehensive, forward-looking career roadmap and industry analysis based on the
learner's validated CBC pathway.
CONTEXT: A Grade 9 graduate and their guardians need long-term visibility into post-secondary routes,
labor demand, and vocational competencies.
CONSTRAINTS:
- Keep language age-appropriate, empowering, and realistic.
- Do not guarantee employment, salary figures, or university admissions.
- Distinguish clearly between academic university degrees and technical/TVET vocational routes.
- Focus on practical Kenyan and regional market realities.
OUTPUT: Return clean, well-formatted Markdown with the following exact section headers:
# CBC Pathway Career Roadmap: {pathway}
## 1. Pathway Overview & Strategic Value
## 2. Key Career Clusters & Professional Roles
## 3. Post-Secondary Education & Training Routes (University & TVET)
## 4. High-Priority Competencies & Practical Skills
## 5. Kenyan Labor Market & Regional Industry Insights
## 6. Recommended Extracurricular & Actionable Next Steps
INPUT:
- Validated Pathway from Stage 1: {pathway}"""
```

8.4 ai_client.py

```
"""
ai_client.py - API communication client with robust error handling.
"""
import os
from openai import OpenAI

MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")

def get_openai_client() -> OpenAI:
    """Validates API credentials and initializes OpenAI client."""
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise RuntimeError("OPENAI_API_KEY is not set. Please add it to your local .env file.")
    return OpenAI(api_key=api_key)

def ask_ai(prompt: str) -> str:
    """
    Dispatches prompt to OpenAI API and safely returns response content.
    Handles connectivity and authentication exceptions gracefully.
    """
    client = get_openai_client()
    try:
        response = client.chat.completions.create(
            model=MODEL,
            messages=[
                {"role": "system", "content": "You are a professional Kenyan CBC education and career
assistant."},
                {"role": "user", "content": prompt}
            ],
            temperature=0.7
        )
```

```
        return response.choices[0].message.content.strip()
    except Exception as exc:
        raise RuntimeError(f"Advisory API communication failed: {exc}") from exc
```

8.5 parsers.py

```
"""
parsers.py - JSON validation and sanitization for Stage 1 outputs.
"""
import json
import re
from typing import Any, Dict

REQUIRED_KEYS = {
    "learner_profile",
    "pathway_alignment",
    "regional_considerations",
    "recommended_school_characteristics",
    "subject_fit_analysis",
    "actionable_next_steps",
    "verification_notes"
}

def parse_school_result(raw_text: str) -> Dict[str, Any]:
    """
    Sanitizes raw AI response and parses it into a verified dictionary.
    Raises ValueError if JSON is corrupted or required schema keys are missing.
    """
    # Clean possible markdown code fences
    cleaned = re.sub(r"^```json\s*", "", raw_text.strip(), flags=re.IGNORECASE)
    cleaned = re.sub(r"```$", "", cleaned.strip())

    try:
        data = json.loads(cleaned)
    except json.JSONDecodeError as exc:
        raise ValueError(f"Model output is not valid JSON: {exc}") from exc

    missing = REQUIRED_KEYS - set(data.keys())
    if missing:
        raise ValueError(f"AI response schema is incomplete. Missing keys: {',
'.join(sorted(missing))}")

    return data
```

8.6 exporter.py

```
"""
exporter.py - Handles disk persistence of advisory session reports.
"""
import json
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, Optional

def export_session(school_result: Optional[Dict[str, Any]], career_report: Optional[str]) -> Path:
    """
    Exports the active consultation session to a structured Markdown document.
    Saves output to an 'exports/' directory with a unique timestamp.
    """
    folder = Path("exports")
    folder.mkdir(exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    file_path = folder / f"cbc_advisory_report_{timestamp}.md"

    content = ["# CBC Senior School & Career Advisory Report", f"*Generated on:
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}*
"]
```

```
    if school_result:
        content.append("## Stage 1: Senior School & Pathway Compatibility Analysis")
        content.append(f"```json
{json.dumps(school_result, indent=2)}
```
")

    if career_report:
        content.append("## Stage 2: Career Opportunity & Industry Roadmap")
        content.append(career_report)

    file_path.write_text("
".join(content), encoding="utf-8")
    return file_path
```

### 8.7 main.py

```
"""
main.py - Entry point and CLI interactive controller for the CBC Advisory Tool.
"""
import os
import sys
from dotenv import load_dotenv

from ai_client import ask_ai
from prompts import school_prompt, career_prompt
from parsers import parse_school_result
from exporter import export_session

load_dotenv()

def prompt_required(label: str) -> str:
    """Prompts user until a non-empty string is provided."""
    while True:
        val = input(f"{label}: ").strip()
        if val:
            return val
        print(" [!] Input cannot be empty. Please provide a valid response.")

def run_stage_one() -> tuple[str, dict]:
    """Executes Stage 1: Senior School Matching & Pathway Alignment."""
    print("
--- STAGE 1: SENIOR SCHOOL MATCHING & PATHWAY ALIGNMENT ---")
    pathway = prompt_required("Enter Preferred CBC Pathway (e.g. STEM - Pure Sciences)")
    location = prompt_required("Enter Preferred County/Location (e.g. Kiambu)")
    subjects = prompt_required("Enter Intended Subject Combinations (e.g. Physics, Chem, Bio, Maths)")

    print("
[*] Analyzing institutional fit and CBC pathway synergy with AI...")
    raw_response = ask_ai(school_prompt(pathway, location, subjects))
    parsed = parse_school_result(raw_response)

    print("
" + "="*60)
    print("STAGE 1 COMPATIBILITY SUMMARY:")
    print(f"Alignment Status : {parsed.get('pathway_alignment', {}).get('status')}")
    print(f"Summary   : {parsed.get('pathway_alignment', {}).get('summary')}")
    print(f"Regional Notes   : {parsed.get('regional_considerations')}")
    print("="*60)
    return pathway, parsed

def run_stage_two(pathway: str) -> str:
    """Executes Stage 2: Career Opportunity & Industry Insights."""
    print(f"
--- STAGE 2: CAREER OPPORTUNITY & INDUSTRY INSIGHTS FOR [{pathway}] ---")
    print("[*] Generating comprehensive career progression roadmap...")
    report = ask_ai(career_prompt(pathway))
    print("
```
```

```
" + "="*60)
    print(report)
    print("="*60)
    return report

def main():
    """Main application loop."""
    school_result = None
    career_report = None
    current_pathway = None

    print("="*65)
    print("  CBC SENIOR SCHOOL SELECTION & CAREER ADVISORY TOOL")
    print("  WeCan Academy - Artificial Intelligence Course (The Lions)")
    print("="*65)

    while True:
        print("
MAIN MENU:")
        print("  1. Senior School Matching & Pathway Alignment (Stage 1)")
        print("  2. Career Opportunity & Industry Insights (Stage 2)")
        print("  3. Export Session & Exit")

        choice = input("
Select an option (1-3): ").strip()

        try:
            if choice == "1":
                current_pathway, school_result = run_stage_one()
            elif choice == "2":
                if not current_pathway:
                    print(" [!] Please complete Stage 1 first to set your pathway profile.")
                    continue
                career_report = run_stage_two(current_pathway)
            elif choice == "3":
                if school_result or career_report:
                    saved_path = export_session(school_result, career_report)
                    print(f"
[+] Session successfully saved to: {saved_path}")
                print("
Thank you for using the CBC Advisory Tool. Best wishes on your academic journey!")
                sys.exit(0)
            else:
                print(" [!] Invalid option. Please select 1, 2, or 3.")
        except Exception as err:
            print(f"
[ERROR] An error occurred during execution: {err}")

if __name__ == "__main__":
    main()
```